



Group Project



Title:

School Administration System using Binary Search Tree (BST)

Subtitle:

Efficient Student Record Management System

Details:

Course: Data Structures

Group Members:

P. Siva Kumar Reddy

C. Karthik Varma





Problem Statement

Schools need an efficient system to manage student data

Manual handling leads to:

Data loss

Slow searching

Errors in records

Need fast operations:

Insert

Search

Update

Delete

Solution: Use Binary Search Tree (BST)





Real-World Application

School/College management systems

Student result processing

Database indexing systems

Administrative tools

Helps in:

Fast searching

Sorted data storage

Efficient updates





Data Structure Used

Binary Search Tree (BST)

Properties:

Left subtree $\rightarrow$ smaller values

Right subtree $\rightarrow$ larger values

Advantages:

Efficient searching

Sorted traversal

Dynamic memory usage





System Design

Main Modules:

Add Student

Display Students

Search Student

Update Student

Delete Student

Count Students

Find Topper

Filter by Marks

Flow:

User Input → Processing → Output





Algorithm (Insert & Search)

Insert:

If root is *NULL* $\rightarrow$ create node

If *ID* < root $\rightarrow$ go left

Else $\rightarrow$ go right

Search:

Compare *ID*

Traverse left/right accordingly

Return result





Algorithm (Delete)

Cases:

No child → delete directly

One child → replace node

Two children →

Find inorder successor

Replace data

Delete successor





Implementation

Language: C Programming

Concepts used:

Structures

Pointers

Dynamic memory (malloc, free)

Modular functions for each operation





Features

Grade calculation

Topper detection

Filter by marks

Menu-driven interface

Memory-safe implementation





Demo Output

Example:

Plain text

Enter Student ID: 101

Enter Name: Ravi

Enter Age: 20

Enter Marks: 85

--- STUDENT RECORDS ---

ID: 101

Name: Ravi

Marks: 85.00

Grade: B

TOPPER DETAILS:

Name: Ravi

Marks: 85





Time Complexity

Operation

Complexity

Insert

$O(\log n)$

Search

$O(\log n)$

Delete

$O(\log n)$

Traversal

$O(n)$





Conclusion

Efficient system using BST

Faster than traditional methods

Suitable for real-world use

Future Scope:

Self-balancing trees (AVL)

GUI interface

Database integration





Thank You

